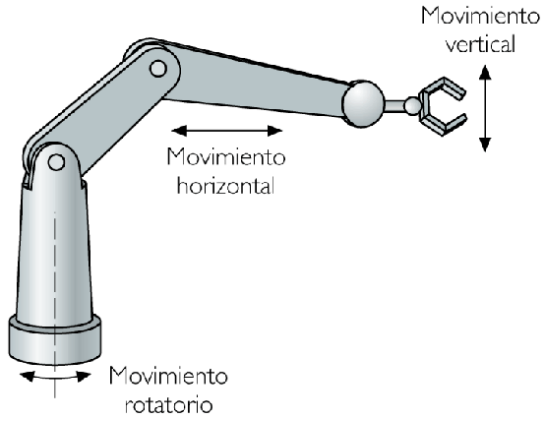


Robótica Teoría

Clase 01: Introducción

Profesor: Carlos Diaz

cdiazd@uni.edu.pe

Contenido Clase 01

- Vectores
- Matrices
- Derivadas
- Jacobiano
- Arco tangente
- Integrales
- Gráficas

Introducción al curso

- Este curso cubre los fundamentos mecánicos y de control de robots, empezando con herramientas matemáticas como matrices de transformación y cuaterniones.
- Incluye modelado cinemático con métodos de Denavit-Hartenberg, análisis de la matriz Jacobiana, y modelado dinámico del robot.
- También aborda estrategias de control cinemático y dinámico para definir y ejecutar trayectorias.
- Se utilizará Python para implementar estos conceptos.

Vectores

- Declarar un vector en numpy.

```
import numpy as np
# Declarar un vector
vector = np.array([1, 2, 3, 4, 5])
print(vector)
```

```
In [1]: runfile('C:/Users/INTEL/Documents/
temp.py', wdir='C:/Users/INTEL/Documents')
[1 2 3 4 5]

In [2]:
```

- Declarar un vector en sympy.

```
import sympy as sp
# Declara las variables simbólicas
x, y, z = sp.symbols("x y z")
# Declara un vector columna
vector = sp.Matrix([x, y, z]) # Para declarar vector fila sp.Matrix([[x, y, z]])
print (vector)
```

```
In [1]: runfile('C:/Users/INTEL/Documents/
temp.py', wdir='C:/Users/INTEL/Documents')
Matrix([[x], [y], [z]])

In [2]:
```

Transpuesta de un vector

- Transpuesta de un vector en numpy.

En numpy, la transposición se aplica típicamente a matrices bidimensionales. Para un vector unidimensional, la transposición no cambia nada porque es esencialmente una sola fila o columna.

$$\boldsymbol{x}^T = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}$$

```
import numpy as np
# Declarar un vector
vector = np.array([1, 2, 3, 4, 5])
# Imprimiendo el vector
print("Vector:")
print(vector)
# Convertir el vector en una matriz columna
matriz_columna = vector.reshape(-1, 1)
print("Matriz columna:")
print(matriz_columna)
# Convertir el vector en una matriz fila
matriz_fila = vector.reshape(1, -1)
print("Matriz fila:")
print(matriz_fila)
```

```
Vector:
[1 2 3 4 5]
Matriz columna:
[[1]
 [2]
 [3]
 [4]
 [5]]
Matriz fila:
[[1 2 3 4 5]]
```

Transpuesta de un vector

- Transpuesta de un vector en sympy.

$$\boldsymbol{x}^T = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}$$

```
import sympy as sp
# Definir los símbolos
x, y, z = sp.symbols("x y z")
# Crear el vector como una matriz columna
vector = sp.Matrix([x, y, z]) # Para crear una matriz fila sp.Matrix([[x, y, z]])
print("Matriz columna:")
print(vector)
# Obtener la transpuesta del vector
vector_T = vector.T
# Imprimir la transpuesta
print("Matriz fila")
print(vector_T)
```

```
Matriz columna:
Matrix([[x], [y], [z]])
Matriz fila
Matrix([[x, y, z]])
```

Producto escalar

- Producto escalar de dos vectores en numpy.

```
import numpy as np
# Definir los vectores
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
# Calcular el producto escalar
dot_producto = np.dot(v1, v2)
# Imprimir el resultado
print(dot_producto)
```

$$\boldsymbol{x} \cdot \boldsymbol{y} = \sum_{i=1}^{n} x_i y_i = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$$

```
In [1]: runfile('C:/Users/INTEL/
Documents/temp.py', wdir='C:/Users/
INTEL/Documents')
32
```

Producto escalar

- Producto escalar de dos vectores en sympy.

```
import sympy as sp
# Definir los símbolos para los componentes de los vectores
x1, y1, z1 = sp.symbols('x1 y1 z1')
x2, y2, z2 = sp.symbols('x2 y2 z2')
# Crear los vectores como matrices columna
v1 = sp.Matrix([x1, y1, z1])
v2 = sp.Matrix([x2, y2, z2])
# Calcular el producto escalar
dot_producto = v1.dot(v2)
print(dot_producto)
# Reemplazar los símbolos con valores numéricos
valores = {x1: 1, y1: 2, z1: 3, x2: 4, y2: 5, z2: 6}
# valores = [(x1, 1), (y1, 2), (z1, 3), (x2, 4), (y2, 5), (z2, 6)]
# valores = [[x1, 1], [y1, 2], [z1, 3], [x2, 4], [y2, 5], [z2, 6]]
dot_producto_evaluado = dot_producto.subs(valores)
# Imprimir el resultado evaluado
print(dot_producto_evaluado)
```

$$\boldsymbol{x} \cdot \boldsymbol{y} = \sum_{i=1}^{n} x_i y_i = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$$

```
x1*x2 + y1*y2 + z1*z2
32
```

Norma de un vector

- Norma de un vector en numpy.

```
import numpy as np

# Define tu vector de 3 dimensiones
vector = np.array([1, 2, 3])

# Calcula la norma del vector
norma = np.linalg.norm(vector)

print("La norma del vector es:", norma)
```

$$\|\boldsymbol{x}\| = \sqrt{\boldsymbol{x} \cdot \boldsymbol{x}} = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

```
La norma del vector es:
3.7416573867739413
```

Norma de un vector

- Norma de un vector en sympy.

$$\|\boldsymbol{x}\| = \sqrt{\boldsymbol{x} \cdot \boldsymbol{x}} = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

```
La norma simbólica del vector es: sqrt(Abs(x)**2 + Abs(y)**2 + Abs(z)**2)
El vector sustituido es: Matrix([[1], [2], [3]])
La norma del vector sustituido es: sqrt(14)
La norma del vector sustituido como número es: 3.74165738677394
```

```
import sympy as sp
# Define las variables simbólicas
x, y, z = sp.symbols('x y z')
# Define el vector
vector = sp.Matrix([x, y, z])
# Calcula la norma simbólica del vector
norma_simb = vector.norm()
# Sustituye los valores en el vector
vector_substituido = vector.subs({x: 1, y: 2, z: 3})
# Calcula la norma del vector con los valores sustituidos
norma_num = vector_substituido.norm()
# Evaluar el resultado numérico
norma_num_evaluado = norma_num.evalf()
print("La norma simbólica del vector es:", norma_simb)
print("El vector sustituido es:", vector_substituido)
print("La norma del vector sustituido es:", norma_num)
print("La norma del vector sustituido como número es:", norma_num_evaluado)
```

Producto vectorial

- Producto vectorial en numpy.

```
import numpy as np
# Define los dos vectores
vector1 = np.array([1, np.sqrt(2), 3])
vector2 = np.array([4, 5, 6])
# Calcula el producto vectorial
producto_vectorial = np.cross(vector1, vector2)
# Redondea el resultado a 2 decimales
producto_vectorial_redondeado = np.round(producto_vectorial, 2)
print("El producto vectorial de", vector1, "y", vector2, "es:\n", producto_vectorial_redondeado)
```

$$\boldsymbol{x} \times \boldsymbol{y} = \begin{bmatrix} x_2 y_3 - x_3 y_2 \\ x_3 y_1 - x_1 y_3 \\ x_1 y_2 - x_2 y_1 \end{bmatrix}$$

```
El producto vectorial de [1.         1.41421356 3.        ] y [4 5 6] es:
 [-6.51  6.   -0.66]
```

Producto vectorial

- Producto vectorial en sympy.

```
El producto vectorial simbólico es: Matrix([[y1*z2 - y2*z1], [-x1*z2 + x2*z1], [x1*y2 - x2*y1]])
El producto vectorial con valores sustituidos es: Matrix([[-15 + 6*sqrt(2)], [6], [5 - 4*sqrt(2)]])
El producto vectorial evaluado es: Matrix([[-6.51471862576143], [6.00000000000000], [-0.656854249492380]])
El producto vectorial evaluado y redondeado a 2 decimales es: [-6.51, 6.0, -0.66]
```

```
import sympy as sp
# Define las variables simbólicas
x1, y1, z1 = sp.symbols('x1 y1 z1')
x2, y2, z2 = sp.symbols('x2 y2 z2')
# Define los dos vectores
vector1 = sp.Matrix([x1, y1, z1])
vector2 = sp.Matrix([x2, y2, z2])
# Calcula el producto vectorial simbólico
producto_vectorial = vector1.cross(vector2)
# Sustituye los valores en los vectores
vector1_substituido = vector1.subs({x1: 1, y1: sp.sqrt(2), z1: 3})
vector2_substituido = vector2.subs({x2: 4, y2: 5, z2: 6})
# Calcula el producto vectorial con los valores sustituidos
producto_vectorial_substituido = vector1_substituido.cross(vector2_substituido)
# Evalúa el resultado numérico
producto_vectorial_evaluado = producto_vectorial_substituido.evalf()
# Redondea los resultados a 2 decimales
producto_vectorial_evaluado_redondeado = [sp.N(val, 2) for val in producto_vectorial_evaluado]
print("El producto vectorial simbólico es:", producto_vectorial)
print("El producto vectorial con valores sustituidos es:", producto_vectorial_substituido)
print("El producto vectorial evaluado es:", producto_vectorial_evaluado)
print("El producto vectorial evaluado y redondeado a 2 decimales es:", producto_vectorial_evaluado_redondeado)
```

Producto exterior

- Producto exterior en numpy.

```
import numpy as np
# Define dos vectores
vector1 = np.array([1, 2, 3])
vector2 = np.array([4, 5, 6])
# Calcula el producto exterior
producto_exterior = np.outer(vector1, vector2)
print("El producto exterior de", vector1, "y", vector2, "es:\n", producto_exterior)
```

$$\boldsymbol{x}\boldsymbol{y}^T = \begin{bmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_n \\ x_2y_1 & x_2y_2 & \cdots & x_2y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_ny_1 & x_ny_2 & \cdots & x_ny_n \end{bmatrix}$$

```
El producto exterior de [1 2 3] y [4 5 6] es:
 [[ 4  5  6]
 [ 8 10 12]
 [12 15 18]]
```

Producto exterior

- Producto exterior en sympy.

$$\boldsymbol{x}\boldsymbol{y}^T = \begin{bmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_n \\ x_2y_1 & x_2y_2 & \cdots & x_2y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_ny_1 & x_ny_2 & \cdots & x_ny_n \end{bmatrix}$$

```
import sympy as sp
# Define las variables simbólicas para dos vectores
x1, x2, x3 = sp.symbols('x1 x2 x3')
y1, y2, y3 = sp.symbols('y1 y2 y3')
# Define los dos vectores
vector1 = sp.Matrix([x1, x2, x3])
vector2 = sp.Matrix([y1, y2, y3])
# Calcula el producto exterior
producto_exterior = vector1 * vector2.T
print("El producto exterior simbólico es:")
print(producto_exterior)
# Sustituye los valores en los vectores
vector1_substituido = vector1.subs({x1: 1, x2: 2, x3: 3})
vector2_substituido = vector2.subs({y1: 4, y2: 5, y3: 6})
# Evalúa el producto exterior con los valores sustituidos
producto_exterior_evaluado = vector1_substituido * vector2_substituido.T
print("\nEl producto exterior evaluado con valores es:")
print(producto_exterior_evaluado)
```

```
El producto exterior simbólico es:
Matrix([[x1*y1, x1*y2, x1*y3], [x2*y1, x2*y2, x2*y3], [x3*y1, x3*y2, x3*y3]])

El producto exterior evaluado con valores es:
Matrix([[4, 5, 6], [8, 10, 12], [12, 15, 18]])
```

Triple producto escalar

- Triple producto escalar en numpy.

```
import numpy as np
# Definir los tres vectores
a = np.array([1, 4, 7])
b = np.array([8, 2, 5])
c = np.array([6, 9, 3])
# Calcular el producto cruzado de b y c
producto_vectorial  = np.cross(b, c)
# Calcular el triple producto escalar (a . (b x c))
triple_producto_escalar  = np.dot(a, producto_vectorial)
print(f"El triple producto escalar es: {triple_producto_escalar}")
```

$$\boldsymbol{x} \cdot (\boldsymbol{y} \times \boldsymbol{z}) = \boldsymbol{y} \cdot (\boldsymbol{z} \times \boldsymbol{x}) = \boldsymbol{z} \cdot (\boldsymbol{x} \times \boldsymbol{y})$$

```
El triple producto escalar es: 405
```

Triple producto escalar

- Triple producto escalar en sympy.

$$x \cdot (y \times z) = y \cdot (z \times x) = z \cdot (x \times y)$$

```
Resultado simbólico del triple producto escalar:
a1*(b2*c3 - b3*c2) + a2*(-b1*c3 + b3*c1) + a3*(b1*c2 - b2*c1)

Resultado numérico del triple producto escalar: 405
```

```
import sympy as sp
# Definir las variables simbólicas
a1, a2, a3 = sp.symbols('a1 a2 a3')
b1, b2, b3 = sp.symbols('b1 b2 b3')
c1, c2, c3 = sp.symbols('c1 c2 c3')
# Definir los tres vectores simbólicos
a = sp.Matrix([a1, a2, a3])
b = sp.Matrix([b1, b2, b3])
c = sp.Matrix([c1, c2, c3])
# Calcular el producto cruzado de b y c
producto_vectorial = b.cross(c)
# Calcular el triple producto escalar (a . (b x c))
triple_producto_escalar = a.dot(producto_vectorial)
# Mostrar el resultado simbólico
print("Resultado simbólico del triple producto escalar:")
print(triple_producto_escalar)
# Reemplazar los valores numéricos
valores = {a1: 1, a2: 4, a3: 7, b1: 8, b2: 2, b3: 5, c1: 6, c2: 9, c3: 3}
triple_producto_escalar_numerico = triple_producto_escalar.subs(valores)
# Mostrar el resultado numérico
print(f"\nResultado numérico del triple producto escalar: {triple_producto_escalar_numerico}")
```

Diferenciación de vectores

- Derivada de un vector en sympy.

```
import sympy as sp
# Definir la variable simbólica
t = sp.symbols('t')
# Definir el vector función f(t)
f_t = sp.Matrix([sp.sin(t), sp.cos(t), t**2])
# Calcular la derivada del vector respecto a t
df_dt = f_t.diff(t)
# Mostrar el resultado simbólico
print("Derivada simbólica del vector f(t) respecto a t:")
print(df_dt)
# Reemplazar valores numéricos en la derivada
valor_t = 2  # Por ejemplo, t = 2
df_dt_numerico = df_dt.subs(t, valor_t)
# Mostrar el resultado numérico
print(f"\nDerivada del vector f(t) en t = {valor_t}:")
print(df_dt_numerico)
# Desarrollar los cálculos numéricos
df_dt_numerico = df_dt_numerico.evalf()
print(f"\nDerivada numérica del vector f(t) en t = {valor_t}, evaluada:")
print(df_dt_numerico)
# Redondear a 2 decimales
df_dt_numerico = sp.Matrix([sp.N(x, 2) for x in df_dt_numerico])
print(f"\nDerivada numérica del vector f(t) en t = {valor_t}, evaluada y redondeada a 2 decimales:")
print(df_dt_numerico)
```

$$\dot{\boldsymbol{x}}(t) = \begin{bmatrix} \dot{x}_1(t) & \dot{x}_2(t) & \cdots & \dot{x}_n(t) \end{bmatrix}$$

```
Derivada simbólica del vector f(t) respecto a t:
Matrix([[cos(t)], [-sin(t)], [2*t]])

Derivada del vector f(t) en t = 2:
Matrix([[cos(2)], [-sin(2)], [4]])

Derivada numérica del vector f(t) en t = 2, evaluada:
Matrix([[-0.416146836547142], [-0.909297426825682], [4.00000000000000]])

Derivada numérica del vector f(t) en t = 2, evaluada y redondeada a 2 decimales:
Matrix([[-0.42], [-0.91], [4.0]])
```

Derivada del producto escalar

- Derivada del producto escalar en sympy.

$$\frac{d}{dt}\boldsymbol{x}\cdot\boldsymbol{y}=\frac{d\boldsymbol{x}}{dt}\cdot\boldsymbol{y}+\boldsymbol{x}\cdot\frac{d\boldsymbol{y}}{dt}$$

```
import sympy as sp
# Definir la variable simbólica
t = sp.symbols('t')
# Definir las dos funciones vectoriales f(t) y g(t)
f_t = sp.Matrix([sp.sin(t), sp.cos(t), t**2])
g_t = sp.Matrix([sp.exp(t), t, sp.log(t + 1)])
# Calcular el producto escalar de f(t) y g(t)
producto_escalar = f_t.dot(g_t)
# Calcular la derivada del producto escalar respecto a t
derivada_producto_escalar = sp.diff(producto_escalar, t)
# Mostrar el resultado simbólico
print("Derivada simbólica del producto escalar de f(t) y g(t) respecto a t:")
print(derivada_producto_escalar)
# Reemplazar valores numéricos en la derivada
valor_t = 2  # Por ejemplo, t = 2
derivada_numerica = derivada_producto_escalar.subs(t, valor_t)
# Mostrar el resultado numérico
print(f"\nDerivada del producto escalar en t = {valor_t}:")
print(derivada_numerica)
# Desarrollar los cálculos numéricos y redondear a 2 decimales
derivada_numerica_redondeada = sp.N(derivada_numerica, 2)
print(f"\nDerivada numérica del producto escalar en t = {valor_t}, evaluada y redondeada a 2 decimales:")
print(derivada_numerica_redondeada)
```

```
Derivada simbólica del producto escalar de f(t) y g(t) respecto a t:
t**2/(t + 1) + 2*t*log(t + 1) - t*sin(t) + exp(t)*sin(t) + exp(t)*cos(t) + cos(t)

Derivada del producto escalar en t = 2:
exp(2)*cos(2) - 2*sin(2) + cos(2) + 4/3 + 4*log(3) + exp(2)*sin(2)

Derivada numérica del producto escalar en t = 2, evaluada y redondeada a 2 decimales:
7.1
```

Derivada del producto vectorial

- Derivada del producto vectorial en sympy.

$$\frac{d}{dt}\boldsymbol{x} \times \boldsymbol{y} = \frac{d\boldsymbol{x}}{dt} \times \boldsymbol{y} + \boldsymbol{x} \times \frac{d\boldsymbol{y}}{dt}$$

```
Producto vectorial simbólico:
Matrix([[-t**3 + log(t + 1)*cos(t)], [t**2*exp(t) - log(t + 1)*sin(t)], [t*sin(t) - exp(t)*cos(t)]])

Derivada del producto vectorial simbólica:
Matrix([[-3*t**2 - log(t + 1)*sin(t) + cos(t)/(t + 1)], [t**2*exp(t) + 2*t*exp(t) - log(t + 1)*cos(t)
- sin(t)/(t + 1)], [t*cos(t) + exp(t)*sin(t) - exp(t)*cos(t) + sin(t)]])

Derivada del producto vectorial numérica (redondeada a 2 decimales):
[-13., 59., 9.9]
```

```
import sympy as sp
# Definir la variable simbólica
t = sp.symbols('t')
# Definir las dos funciones vectoriales f(t) y g(t)
f_t = sp.Matrix([sp.sin(t), sp.cos(t), t**2])
g_t = sp.Matrix([sp.exp(t), t, sp.log(t + 1)])
# Producto vectorial
producto_vectorial = f_t.cross(g_t)
# Derivada del producto vectorial
derivada_producto_vectorial = producto_vectorial.diff(t)
# Valores numéricos de t
t_val = 2  # Puedes ajustar el valor de t según sea necesario
valores_numericos = {t: t_val}
# Evaluar la derivada del producto vectorial en valores numéricos
derivada_producto_vectorial_numerico = derivada_producto_vectorial.subs(valores_numericos)
# Redondear a 2 decimales usando sp.N
derivada_producto_vectorial_numerico_redondeado = [sp.N(val, 2) for val in derivada_producto_vectorial_numerico]
# Imprimir resultados
print("Producto vectorial simbólico:")
print(producto_vectorial)
print("\nDerivada del producto vectorial simbólica:")
print(derivada_producto_vectorial)
print("\nDerivada del producto vectorial numérica (redondeada a 2 decimales):")
print(derivada_producto_vectorial_numerico_redondeado)
```

Gradiente de una función escalar

- Gradiente de una función escalar en sympy.

$$\nabla_x f(x) = \left[\frac{\partial f(x)}{\partial x_1} \quad \frac{\partial f(x)}{\partial x_2} \quad \cdots \quad \frac{\partial f(x)}{\partial x_n}\right]$$

```
import sympy as sp
# Definir las variables simbólicas
x, y = sp.symbols('x y')
# Definir la función escalar
f = 2*x**3 + y**4
# Definir el vector de variables
variables = sp.Matrix([x, y])
# Calcular el gradiente
gradiente = sp.Matrix([sp.diff(f, var) for var in variables])
# Imprimir el resultado simbólico
print("Función escalar:", f)
print("Gradiente simbólico:", gradiente)
# Evaluar el gradiente en un punto específico, por ejemplo (1, 2)
punto = {x: 1, y: 2}
gradiente_en_punto = gradiente.subs(punto)
# Imprimir el gradiente evaluado numéricamente
print("Gradiente en el punto (1, 2):", gradiente_en_punto)
```

```
Función escalar: 2*x**3 + y**4
Gradiente simbólico: Matrix([[6*x**2], [4*y**3]])
Gradiente en el punto (1, 2): Matrix([[6], [32]])
```

Proyección de un vector sobre otro

- Proyección de un vector sobre otro en numpy.

```
import numpy as np
# Definir los vectores
a = np.array([3, 4])
b = np.array([1, 2])
# Calcular el producto punto de a y b
producto_escalar = np.dot(a, b)
# Calcular la norma de b
norma_b = np.linalg.norm(b)
# Calcular la proyección de a sobre b
proyeccion = (producto_escalar / norma_b**2) * b
# Imprimir el resultado
print("Vector a:", a)
print("Vector b:", b)
print("Proyección de a sobre b:", proyeccion)
```

$$\text{proj}_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|^2}\mathbf{b}$$

```
Vector a: [3 4]
Vector b: [1 2]
Proyección de a sobre b: [2.2 4.4]
```

Proyección de un vector sobre otro

- Proyección de un vector sobre otro en sympy.

```
import sympy as sp
# Definir las variables simbólicas
x, y = sp.symbols('x y')  # Componentes del vector a
u, v = sp.symbols('u v')  # Componentes del vector b
# Definir los vectores simbólicos
a = sp.Matrix([x, y])
b = sp.Matrix([u, v])
# Calcular el producto punto de a y b
producto_escalar = a.dot(b)
# Calcular la norma de b
norma_b = b.norm()  # Norma de b
# Calcular la proyección simbólica de a sobre b
proyeccion_simbólica = (producto_escalar / norma_b**2) * b
# Reemplazar valores numéricos en a y b
valores = {x: 3, y: 4, u: 1, v: 2}
proyeccion_numerica = proyeccion_simbólica.subs(valores)
# Redondear a 2 decimales usando sp.N
proyeccion_numerica_redondeada = [sp.N(comp, 2) for comp in proyeccion_numerica]
# Imprimir resultados
print("Vector simbólico a:", a)
print("Vector simbólico b:", b)
print("Proyección simbólica de a sobre b:\n", proyeccion_simbólica)
print("Proyección numérica de a sobre b con a = [3, 4] y b = [1, 2]:\n", proyeccion_numerica_redondeada)
```

$$\text{proj}_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{b}\|^2}\mathbf{b}$$

```
Vector simbólico a: Matrix([[x], [y]])
Vector simbólico b: Matrix([[u], [v]])
Proyección simbólica de a sobre b:
 Matrix([[u*(u*x + v*y)/(Abs(u)**2 + Abs(v)**2)], [v*(u*x + v*y)/(Abs(u)**2 + Abs(v)**2)]])
Proyección numérica de a sobre b con a = [3, 4] y b = [1, 2]:
 [2.2, 4.4]
```

Matrices

- Declarar una matriz en numpy.

```
import numpy as np
# Declarar una matriz 2x3 (2 filas y 3 columnas)
matriz = np.array([[1, 2, 3], [4, 5, 6]])
# Imprimir la matriz
print("Matriz:")
print(matriz)
# Imprimir las dimensiones de la matriz
print("Dimensiones de la matriz:", matriz.shape)
```

```
Matriz:
[[1 2 3]
 [4 5 6]]
Dimensiones de la matriz: (2, 3)
```

- Declarar una matriz en sympy.

```
import sympy as sp
# Definir las variables simbólicas
a, b, c, d, e, f = sp.symbols("a b c d e f")
# Declarar una matriz 2x2 con variables simbólicas
matriz = sp.Matrix([[a, b, c], [d, e, f]])
# Imprimir la matriz
print("Matriz:")
print(matriz)
# Imprimir las dimensiones de la matriz
print("Dimensiones de la matriz:", matriz.shape)
```

```
Matriz:
Matrix([[a, b, c], [d, e, f]])
Dimensiones de la matriz: (2, 3)
```

Transpuesta de una matriz

- Transpuesta de una matriz en numpy.

```
import numpy as np
# Declarar una matriz 2x3 (2 filas y 3 columnas)
matriz = np.array([[1, 2, 3], [4, 5, 6]])
# Obtener la transpuesta de la matriz
matriz_transpuesta = matriz.T
# Imprimir la matriz original
print("Matriz original:")
print(matriz)
# Imprimir la matriz transpuesta
print("Matriz transpuesta:")
print(matriz_transpuesta)
```

$$(A^T)_{ij} = A_{ji}$$

```
Matriz original:
[[1 2 3]
 [4 5 6]]
Matriz transpuesta:
[[1 4]
 [2 5]
 [3 6]]
```

Transpuesta de una matriz

- Transpuesta de una matriz en sympy.

```
import sympy as sp
# Definir una matriz simbólica
a, b, c, d = sp.symbols('a b c d')
matriz = sp.Matrix([[a, b], [c, d]])
# Obtener la transpuesta de la matriz
matriz_transpuesta = matriz.T
# Imprimir la matriz original
print("Matriz original:")
print(matriz)
# Imprimir la matriz transpuesta
print("Matriz transpuesta:")
print(matriz_transpuesta)
```

$$(A^T)_{ij} = A_{ji}$$

```
Matriz original:
Matrix([[a, b], [c, d]])
Matriz transpuesta:
Matrix([[a, c], [b, d]])
```

Matriz triangular superior e inferior

- Matriz triangular superior e inferior en numpy.

```
import numpy as np
# Crear una matriz 3x3
matriz = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
# Obtener la matriz triangular superior
matriz_triang_sup = np.triu(matriz)
# Obtener la matriz triangular inferior
matriz_triang_inf = np.tril(matriz)
# Imprimir la matriz original y la triangular superior e inferior
print("Matriz original:")
print(matriz)
print("\nMatriz triangular superior:")
print(matriz_triang_sup)
print("\nMatriz triangular inferior:")
print(matriz_triang_inf)
```

```
Matriz original:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Matriz triangular superior:
[[1 2 3]
 [0 5 6]
 [0 0 9]]

Matriz triangular inferior:
[[1 0 0]
 [4 5 0]
 [7 8 9]]
```

Matriz triangular superior e inferior

- Matriz triangular superior e inferior en sympy.

```
import sympy as sp
# Definir una matriz simbólica 3x3
a, b, c, d, e, f, g, h, i = sp.symbols('a b c d e f g h i')
matriz = sp.Matrix([[a, b, c],
                    [d, e, f],
                    [g, h, i]])
# Obtener la matriz triangular superior
matriz_triang_sup = matriz.upper_triangular()
# Obtener la matriz triangular inferior
matriz_triang_inf = matriz.lower_triangular()
# Imprimir la matriz original y la triangular superior
print("Matriz original:")
sp.pprint(matriz)
print("\nMatriz triangular superior:")
sp.pprint(matriz_triang_sup)
print("\nMatriz triangular inferior:")
sp.pprint(matriz_triang_inf)
```

```
Matriz original:
⎡a  b  c⎤
⎢       ⎥
⎢d  e  f⎥
⎢       ⎥
⎣g  h  i⎦

Matriz triangular superior:
⎡a  b  c⎤
⎢       ⎥
⎢0  e  f⎥
⎢       ⎥
⎣0  0  i⎦

Matriz triangular inferior:
⎡a  0  0⎤
⎢       ⎥
⎢d  e  0⎥
⎢       ⎥
⎣g  h  i⎦
```

Matriz diagonal

- Matriz diagonal en numpy.

```
import numpy as np
# Crear una matriz diagonal con elementos 1, 2, 3
diagonal = np.diag([1, 2, 3])
# Imprimir la matriz diagonal
print("Matriz Diagonal:")
print(diagonal)
```

```
Matriz Diagonal:
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

- Matriz diagonal en sympy.

```
import sympy as sp
# Definir variables simbólicas
a, b, c = sp.symbols('a b c')
# Crear una matriz diagonal con las variables simbólicas
matriz_diagonal = sp.diag(a, b, c)
# Imprimir la matriz diagonal
print("Matriz Diagonal:")
sp.pprint(matriz_diagonal)
```

```
Matriz Diagonal:
⎡a  0  0⎤
⎢       ⎥
⎢0  b  0⎥
⎢       ⎥
⎣0  0  c⎦
```

Matriz identidad

- Matriz identidad en numpy.

```
import numpy as np
# Crear una matriz identidad 4x4
matriz_identidad = np.eye(4)
# Imprimir la matriz identidad
print("Matriz Identidad:")
print(matriz_identidad)
```

```
Matriz Identidad:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

- Matriz identidad en sympy.

```
import sympy as sp
# Crear una matriz identidad de tamaño 3x3
matriz_identidad = sp.eye(4)
# Imprimir la matriz identidad
print("Matriz Identidad:")
sp.pprint(matriz_identidad)
```

```
Matriz Identidad:
⎡1  0  0  0⎤
⎢          ⎥
⎢0  1  0  0⎥
⎢          ⎥
⎢0  0  1  0⎥
⎢          ⎥
⎣0  0  0  1⎦
```

Igualdad de matrices

- Igualdad de matrices en numpy.

```
import numpy as np
# Crear dos matrices
matriz1 = np.array([[1, 2, 3],
                    [4, 5, 6],
                    [7, 8, 9]])

matriz2 = np.array([[1, 2, 3],
                    [4, 5, 6],
                    [7, 8, 9]])
# Verificar la igualdad
son_iguales = np.array_equal(matriz1, matriz2)
# Imprimir el resultado
print("¿Son las matrices iguales?:", son_iguales)
# Verificar la igualdad elemento a elemento
son_iguales_elementos = matriz1==matriz2
# Imprimir el resultado
print("¿Los elementos son iguales?:\n", son_iguales_elementos)
```

```
¿Son las matrices iguales?: True
¿Los elementos son iguales?:
 [[ True  True  True]
 [ True  True  True]
 [ True  True  True]]
```

Igualdad de matrices

- Igualdad de matrices en sympy.

```
import sympy as sp
# Crear dos matrices simbólicas
matriz1 = sp.Matrix([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])
matriz2 = sp.Matrix([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])
# Verificar la igualdad
son_iguales = matriz1 == matriz2
# Imprimir el resultado
print("¿Son las matrices iguales?:", son_iguales)
```

```
¿Son las matrices iguales?: True
```

Multiplicación de Matrices

- Multiplicación de matrices en numpy.

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

```
import numpy as np
# Definir dos matrices
matriz1 = np.array([[1, 2, 3],
                    [4, 5, 6]])
matriz2 = np.array([[7, 8],
                    [9, 10],
                    [11, 12]])
# Multiplicar las matrices
producto_matrices = np.dot(matriz1, matriz2)
# También puede usar producto_matrices = matriz1 @ matriz2
# No se usa * porque multiplica elemento a elemento y no un producto matricial
# Imprimir el resultado
print("Producto de las matrices:")
print(producto_matrices)
```

```
Producto de las matrices:
[[ 58  64]
 [139 154]]
```

Multiplicación de Matrices

- Multiplicación de matrices en sympy.

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

```
import sympy as sp
# Definir variables simbólicas
a, b, c, d, e, f, g, h = sp.symbols('a b c d e f g h')
# Crear dos matrices simbólicas
matriz1 = sp.Matrix([[a, b],
                     [c, d]])
matriz2 = sp.Matrix([[e, f],
                     [g, h]])
# Multiplicar las matrices. No existe dot en sympy
producto_matrices = matriz1 * matriz2
# Definir valores para las variables
valores = {a: 1, b: 2, c: 3, d: 4, e: 5, f: 6, g: 7, h: 8}
# Evaluar el producto con los valores numéricos
producto_matrices_evaluado = producto_matrices.subs(valores)
# Imprimir el resultado simbólico
print("Producto de las matrices simbólicas:")
sp.pprint(producto_matrices)
# Imprimir el resultado numérico
print("Producto de las matrices evaluado numéricamente:")
print(producto_matrices_evaluado)
```

```
Producto de las matrices simbólicas:
⎡a·e + b·g  a·f + b·h⎤
⎢                    ⎥
⎣c·e + d·g  c·f + d·h⎦
Producto de las matrices evaluado numéricamente:
Matrix([[19, 22], [43, 50]])
```

Traza de una matriz

- Traza de una matriz en numpy.

```
import numpy as np
# Definir una matriz
matriz = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
# Calcular la traza de la matriz
traza = np.trace(matriz)
# Imprimir el resultado
print("Matriz:")
print(matriz)
print("Traza de la matriz:", traza)
```

$$\mathrm{tr}(A) = \sum_{i=1}^{n} a_{ii}$$

```
Matriz:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Traza de la matriz: 15
```

Traza de una matriz

- Traza de una matriz en sympy.

```
import sympy as sp
# Definir variables simbólicas
a, b, c, d, e, f, g, h, i = sp.symbols("a b c d e f g h i")
# Crear una matriz simbólica
matriz = sp.Matrix([[a, b, c],
                    [d, e, f],
                    [g, h, i]])
# Calcular la traza de la matriz
traza = matriz.trace()
# Imprimir el resultado
print("Matriz simbólica:")
sp.pprint(matriz)
print("Traza de la matriz simbólica:")
print(traza)
```

$$\mathrm{tr}(A) = \sum_{i=1}^{n} a_{ii}$$

```
Matriz simbólica:
⎡a  b  c⎤
⎢       ⎥
⎢d  e  f⎥
⎢       ⎥
⎣g  h  i⎦
Traza de la matriz simbólica:
a + e + i
```

Determinante de una matriz

- Determinante de una matriz en numpy.

```
import numpy as np
# Definir una matriz
matriz = np.array([[1, 4, 6],
                   [7, 2, 5],
                   [9, 8, 3]])
# Calcular el determinante de la matriz
determinante = np.linalg.det(matriz)
# Imprimir el resultado
print("Matriz:")
print(matriz)
print("Determinante de la matriz:", round(determinante,2))
```

```
Matriz:
[[1 4 6]
 [7 2 5]
 [9 8 3]]
Determinante de la matriz: 290.0
```

Determinante de una matriz

- Determinante de una matriz en sympy.

```
import sympy as sp
# Definir variables simbólicas
a, b, c, d, e, f, g, h, i = sp.symbols('a b c d e f g h i')
# Crear una matriz simbólica
matriz = sp.Matrix([[a, b, c],
                    [d, e, f],
                    [g, h, i]])
# Calcular el determinante de la matriz
determinante = matriz.det()
# Imprimir el resultado
print("Matriz simbólica:")
sp.pprint(matriz)
print("Determinante de la matriz simbólica:")
print(determinante)
```

```
Matriz simbólica:
⎡a  b  c⎤
⎢       ⎥
⎢d  e  f⎥
⎢       ⎥
⎣g  h  i⎦
Determinante de la matriz simbólica:
a*e*i - a*f*h - b*d*i + b*f*g + c*d*h - c*e*g
```

Norma de una matriz

- Norma de una matriz en numpy.

```
import numpy as np
# Definir una matriz no cuadrada (2x3)
A = np.array([[1, 2, 3],
              [4, 5, 6]])
# Calcular la norma de Frobenius
norma_frobenius = np.linalg.norm(A)
print("Norma de Frobenius:", norma_frobenius)
```

$$\|A\| = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2} = \sqrt{\mathrm{tr}(A^T A)}$$

```
Norma de Frobenius: 9.539392014169456
```

Norma de una matriz

- Norma de una matriz en sympy.

$$\|A\| = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2} = \sqrt{\operatorname{tr}(A^T A)}$$

```
import sympy as sp
# Definir variables simbólicas
a, b, c, d, e, f = sp.symbols('a b c d e f')
# Definir una matriz simbólica no cuadrada (2x3)
A = sp.Matrix([[a, b, c],
               [d, e, f]])
# Calcular la norma de Frobenius automáticamente
norma_frobenius_simbolica = A.norm('fro')
# Mostrar el resultado simbólico
print("Norma de Frobenius (simbólica):", norma_frobenius_simbolica)
# Reemplazar valores y calcular el resultado numérico
valores = {a: 1, b: 2, c: 3, d: 4, e: 5, f: 6}
norma_frobenius_numerica = norma_frobenius_simbolica.subs(valores).evalf()
# Mostrar el resultado numérico
print("Norma de Frobenius (numérica):", norma_frobenius_numerica)
```

```
Norma de Frobenius (simbólica):
 sqrt(Abs(a)**2 + Abs(b)**2 + Abs(c)**2 + Abs(d)**2 + Abs(e)**2 + Abs(f)**2)
Norma de Frobenius (numérica): 9.53939201416946
```

Inversa de una matriz

- Inversa de una matriz en numpy.

```
import numpy as np
# Definir una matriz
matriz = np.array([[1, 2], [3, 4]])
# Calcular la inversa de la matriz
inversa = np.linalg.inv(matriz)
# Multiplicando las matrices
producto=np.dot(matriz,inversa)
# Mostrar la matriz
print("Matriz:")
print(matriz)
# Mostrar la matriz inversa
print("Matriz inversa:")
print(inversa)
# Mostrar demostracion
print("Demostración:")
print(np.round(producto,2))
```

$$AA^{-1} = A^{-1}A = I$$

```
Matriz:
[[1 2]
 [3 4]]
Matriz inversa:
[[-2.   1. ]
 [ 1.5 -0.5]]
Demostración:
[[1. 0.]
 [0. 1.]]
```

Inversa de una matriz

- Inversa de una matriz en sympy.

$$AA^{-1} = A^{-1}A = I$$

```
import sympy as sp
# Definir variables simbólicas
a, b, c, d = sp.symbols('a b c d')
# Definir una matriz simbólica
matriz = sp.Matrix([[a, b], [c, d]])
# Calcular la inversa de la matriz simbólica
inversa_simbolica = matriz.inv()
# Reemplazar valores en la matriz y su inversa
valores = {a: 1, b: 2, c: 3, d: 4}
matriz_numerica = matriz.subs(valores)
inversa_numerica = inversa_simbolica.subs(valores)
# Multiplicar las matrices (simbólica y numérica)
producto_simbolico = matriz * inversa_simbolica
producto_numerico = matriz_numerica * inversa_numerica
# Mostrar la matriz original, la inversa y la demostración
print("Matriz simbólica:")
sp.pprint(matriz)
print("\nMatriz inversa simbólica:")
sp.pprint(inversa_simbolica)
print("\nDemostración simbólica:")
sp.pprint(producto_simbolico)
print("\nMatriz numérica:")
sp.pprint(matriz_numerica)
print("\nMatriz inversa numérica:")
sp.pprint(inversa_numerica)
print("\nDemostración numérica:")
sp.pprint(producto_numerico)
```

```
Matriz simbólica:
⎡a  b⎤
⎢    ⎥
⎣c  d⎦

Matriz inversa simbólica:
⎡    d          -b    ⎤
⎢─────────  ─────────⎥
⎣a⋅d - b⋅c  a⋅d - b⋅c⎦

⎡   -c          a    ⎤
⎢─────────  ─────────⎥
⎣a⋅d - b⋅c  a⋅d - b⋅c⎦

Demostración simbólica:
⎡   a⋅d          b⋅c                          ⎤
⎢───────── - ─────────          0             ⎥
⎣a⋅d - b⋅c   a⋅d - b⋅c                        ⎦

⎡                            a⋅d          b⋅c  ⎤
⎢          0              ─────── - ─────── ⎥
⎣                         a⋅d - b⋅c  a⋅d - b⋅c⎦
```

```
Matriz numérica:
⎡1  2⎤
⎢    ⎥
⎣3  4⎦

Matriz inversa numérica:
⎡-2    1  ⎤
⎢         ⎥
⎣3/2  -1/2⎦

Demostración numérica:
⎡1  0⎤
⎢    ⎥
⎣0  1⎦
```

Exponencial de una matriz

- Exponencial de una matriz en numpy. Numpy no tiene una función exponencial de una matriz y por eso usamos la librería scipy.

```
import numpy as np
import scipy as sc
# SciPy es una biblioteca en Python utilizada para
# la computación científica y técnica.
# Definir una matriz 2x2
matriz = np.array([[1, 2], [3, 4]])
# Calcular la exponencial de la matriz
exponencial = sc.linalg.expm(matriz)
# Mostrar la matriz original y su exponencial
print("Matriz:")
print(matriz)
print("\nExponencial de la matriz:")
print(exponencial)
```

$$e^A = \sum_{n=0}^{n=\infty} \frac{A^n}{n!} = 1 + A + \frac{A^2}{2!} + \cdots + \frac{A^n}{n!}$$

```
Matriz:
[[1 2]
 [3 4]]

Exponencial de la matriz:
[[ 51.9689562   74.73656457]
 [112.10484685 164.07380305]]
```

Exponencial de una matriz

- Si no desea usar scipy puede usar sympy y evaluar la exponencial.

```
import sympy as sp
# Definir la matriz simbólica
A = sp.Matrix([[1, 2], [3, 4]])
# Calcular la exponencial de la matriz
exp_A = A.exp().evalf()
# Mostrar la matriz original y su exponencial
print("Matriz:")
sp.pprint(A)
print("\nExponencial de la matriz:")
sp.pprint(exp_A)
```

$$e^A = \sum_{n=0}^{n=\infty} \frac{A^n}{n!} = 1 + A + \frac{A^2}{2!} + \cdots + \frac{A^n}{n!}$$

```
Matriz:
⎡1  2⎤
⎢    ⎥
⎣3  4⎦

Exponencial de la matriz:
⎡51.968956198705   74.7365645670032⎤
⎢                                  ⎥
⎣112.104846850505  164.07380304921 ⎦
```

Exponencial de una matriz

- Exponencial de una matriz en sympy.

```
import sympy as sp
# Definir la matriz simbólica con variables simbólicas
a, b, c, d = sp.symbols('a b c d')
A = sp.Matrix([[a, b], [c, d]])
# Calcular la exponencial de la matriz de forma simbólica
exp_A_simbolico = A.exp()
# Mostrar la matriz original y su exponencial simbólica
print("Matriz original simbólica:")
print(A)
print("\nExponencial simbólica de la matriz:")
print(exp_A_simbolico)
# Reemplazar valores simbólicos por valores numéricos
valores = {a: 1, b: 2, c: 3, d: 4}
exp_A_numerico = exp_A_simbolico.subs(valores)
# Mostrar el resultado numérico
print("\nExponencial de la matriz con valores numéricos:")
sp.pprint(exp_A_numerico.evalf())
```

$$e^A = \sum_{n=0}^{n=\infty} \frac{A^n}{n!} = 1 + A + \frac{A^2}{2!} + \cdots + \frac{A^n}{n!}$$

```
Matriz original simbólica:
Matrix([[a, b], [c, d]])

Exponencial simbólica de la matriz:
Matrix([[2*b*c*exp(a/2 + d/2 - sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/((a - d + sqrt(a**2 - 2*a*d + 4*b*c +
d**2))*sqrt(a**2 - 2*a*d + 4*b*c + d**2)) - 2*b*c*exp(a/2 + d/2 + sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/((a -
d - sqrt(a**2 - 2*a*d + 4*b*c + d**2))*sqrt(a**2 - 2*a*d + 4*b*c + d**2)), -4*b**2*c*exp(a/2 + d/2 +
sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/((a - d - sqrt(a**2 - 2*a*d + 4*b*c + d**2))*(a**2 - 2*a*d + a*sqrt(a**2
- 2*a*d + 4*b*c + d**2) + 4*b*c + d**2 - d*sqrt(a**2 - 2*a*d + 4*b*c + d**2))) - 2*b*(a**2 - 2*a*d +
a*sqrt(a**2 - 2*a*d + 4*b*c + d**2) + 2*b*c + d**2 - d*sqrt(a**2 - 2*a*d + 4*b*c + d**2))*exp(a/2 + d/2 -
sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/((a - d + sqrt(a**2 - 2*a*d + 4*b*c + d**2))*(a**2 - 2*a*d + a*sqrt(a**2
- 2*a*d + 4*b*c + d**2) + 4*b*c + d**2 - d*sqrt(a**2 - 2*a*d + 4*b*c + d**2)))], [-c*exp(a/2 + d/2 -
sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/sqrt(a**2 - 2*a*d + 4*b*c + d**2) + c*exp(a/2 + d/2 + sqrt(a**2 - 2*a*d
+ 4*b*c + d**2)/2)/sqrt(a**2 - 2*a*d + 4*b*c + d**2), 2*b*c*exp(a/2 + d/2 + sqrt(a**2 - 2*a*d + 4*b*c +
d**2)/2)/(a**2 - 2*a*d + a*sqrt(a**2 - 2*a*d + 4*b*c + d**2) + 4*b*c + d**2 - d*sqrt(a**2 - 2*a*d + 4*b*c +
d**2)) + (a**2 - 2*a*d + a*sqrt(a**2 - 2*a*d + 4*b*c + d**2) + 2*b*c + d**2 - d*sqrt(a**2 - 2*a*d + 4*b*c +
d**2))*exp(a/2 + d/2 - sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/(a**2 - 2*a*d + a*sqrt(a**2 - 2*a*d + 4*b*c +
d**2) + 4*b*c + d**2 - d*sqrt(a**2 - 2*a*d + 4*b*c + d**2))]])

Exponencial de la matriz con valores numéricos:
⎡51.968956198705   74.7365645670032⎤
⎢                                  ⎥
⎣112.104846850505  164.07380304921 ⎦
```

Diferenciación de una matriz

- Rango de una matriz en sympy.

```
import sympy as sp
# Definir la variable simbólica tiempo t
t = sp.symbols('t')
# Definir una matriz simbólica dependiente del tiempo t
A = sp.Matrix([[t**2, sp.sin(t)], [sp.exp(t), t]])
# Diferenciar la matriz con respecto a t
A_derivada = A.diff(t)
# Mostrar la matriz original y su derivada simbólica
print("Matriz original:")
sp.pprint(A)
print("\nMatriz derivada respecto al tiempo:")
sp.pprint(A_derivada)
# Reemplazar el tiempo por t = 2
A_derivada_numerica = A_derivada.subs(t, 2)
# Mostrar el resultado numérico
print("\nMatriz derivada con t = 2:")
sp.pprint(A_derivada_numerica.evalf())
```

$$\dot{A}(t) = \begin{bmatrix} \dot{a}_{11}(t) & \dot{a}_{12}(t) & \cdots & \dot{a}_{1n}(t) \\ \dot{a}_{21}(t) & \dot{a}_{22}(t) & \cdots & \dot{a}_{2n}(t) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{a}_{m1}(t) & \dot{a}_{m2}(t) & \cdots & \dot{a}_{mn}(t) \end{bmatrix}$$

```
Matriz original:
⎡ 2        ⎤
⎢t   sin(t)⎥
⎢          ⎥
⎢ t        ⎥
⎣ℯ     t   ⎦

Matriz derivada respecto al tiempo:
⎡2⋅t  cos(t)⎤
⎢           ⎥
⎢ t         ⎥
⎣ℯ      1   ⎦

Matriz derivada con t = 2:
⎡      4.0          -0.416146836547142⎤
⎢                                     ⎥
⎣7.38905609893065         1.0         ⎦
```

Derivada de la inversa de una matriz

- Derivada de la inversa de una matriz en sympy.

```
import sympy as sp
# Definir la variable simbólica tiempo t
t = sp.symbols('t')
# Definir una matriz simbólica dependiente del tiempo t
A = sp.Matrix([[t, sp.sin(t)], [sp.exp(t), t**2]])
# Calcular la inversa de la matriz A
A_inv = A.inv()
# Calcular la derivada de la matriz A con respecto al tiempo t
A_derivada = A.diff(t)
# Aplicar la regla de la derivada de la inversa de una matriz
A_inv_derivada = -A_inv * A_derivada * A_inv
# Mostrar la matriz inversa y su derivada simbólica
print("Inversa de la matriz:")
print(A_inv)
print("\nDerivada simbólica de la inversa de la matriz:")
print(A_inv_derivada)
# Reemplazar el tiempo t por un valor numérico, por ejemplo, t = 1
A_inv_derivada_numerica = A_inv_derivada.subs(t, 1)
# Mostrar el resultado numérico
print("\nDerivada de la inversa de la matriz con t = 1:")
sp.pprint(A_inv_derivada_numerica.evalf())
```

$$\frac{d}{dt}(A^{-1}) = -A^{-1} \cdot \frac{dA}{dt} \cdot A^{-1}$$

```
Inversa de la matriz:
Matrix([[t**2*exp(t)/(t**3*exp(t) - exp(2*t)*sin(t)), -exp(t)*sin(t)/(t**3*exp(t) -
exp(2*t)*sin(t))], [exp(t)/(-t**3 + exp(t)*sin(t)), -t/(-t**3 + exp(t)*sin(t))]])

Derivada simbólica de la inversa de la matriz:
Matrix([[t**2*(-t**2*exp(t)/(t**3*exp(t) - exp(2*t)*sin(t)) + exp(2*t)*sin(t)/
(t**3*exp(t) - exp(2*t)*sin(t)))*exp(t)/(t**3*exp(t) - exp(2*t)*sin(t)) + (-
t**2*exp(t)*cos(t)/(t**3*exp(t) - exp(2*t)*sin(t)) + 2*t*exp(t)*sin(t)/(t**3*exp(t)
- exp(2*t)*sin(t)))*exp(t)/(-t**3 + exp(t)*sin(t)), -t*(-t**2*exp(t)*cos(t)/
(t**3*exp(t) - exp(2*t)*sin(t)) + 2*t*exp(t)*sin(t)/(t**3*exp(t) -
exp(2*t)*sin(t)))/(-t**3 + exp(t)*sin(t)) - (-t**2*exp(t)/(t**3*exp(t) -
exp(2*t)*sin(t)) + exp(2*t)*sin(t)/(t**3*exp(t) - exp(2*t)*sin(t)))*exp(t)*sin(t)/
(t**3*exp(t) - exp(2*t)*sin(t))], [t**2*(t*exp(t)/(-t**3 + exp(t)*sin(t)) - exp(t)/
(-t**3 + exp(t)*sin(t)))*exp(t)/(t**3*exp(t) - exp(2*t)*sin(t)) + (2*t**2/(-t**3 +
exp(t)*sin(t)) - exp(t)*cos(t)/(-t**3 + exp(t)*sin(t)))*exp(t)/(-t**3 +
exp(t)*sin(t)), -t*(2*t**2/(-t**3 + exp(t)*sin(t)) - exp(t)*cos(t)/(-t**3 +
exp(t)*sin(t)))/(-t**3 + exp(t)*sin(t)) - (t*exp(t)/(-t**3 + exp(t)*sin(t)) -
exp(t)/(-t**3 + exp(t)*sin(t)))*exp(t)*sin(t)/(t**3*exp(t) - exp(2*t)*sin(t))]])

Derivada de la inversa de la matriz con t = 1:
[-1.09737482974303   0.0358221979500285]
[                                       ]
[0.871449869897757   -0.320588491146913]
```

El Jacobiano

- Derivada de la inversa de una matriz en sympy.

```
import sympy as sp
# Definir las variables
x, y, z = sp.symbols('x y z')
# Definir un vector de funciones
F = sp.Matrix([x**2 * y, y * z, sp.sin(x) * z])
# Definir el vector de variables independientes
variables = sp.Matrix([x, y, z])
# Calcular el Jacobiano
J = F.jacobian(variables)
# Imprimir el Jacobiano simbólico
print("Jacobiano simbólico:")
sp.pprint(J)
# Sustituir las variables por valores
valores = {x: 1, y: 2, z: 3}
# Evaluar el Jacobiano con los valores dados
J_numerico = J.subs(valores)
# Imprimir el Jacobiano numérico
print("\nJacobiano numérico con x=1, y=2, z=3:")
sp.pprint(J_numerico)
# Mostrar el Jacobiano numérico evaluado como float
print("\nJacobiano numérico (float):")
J_numerico = J_numerico.evalf()
sp.pprint(J_numerico)
```

Para una función vectorial $\mathbf{F}(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_n(\mathbf{x})]$, donde $\mathbf{x} = [x_1, x_2, \ldots, x_m]$, el Jacobiano es una matriz de tamaño $n \times m$ dada por:

$$J(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_m} \end{bmatrix}$$

```
Jacobiano simbólico:
⎡           2        ⎤
⎢ 2⋅x⋅y    x     0   ⎥
⎢                     ⎥
⎢   0       z     y   ⎥
⎢                     ⎥
⎣z⋅cos(x)   0   sin(x)⎦

Jacobiano numérico con x=1, y=2, z=3:
⎡   4      1    0   ⎤
⎢                   ⎥
⎢   0      3    2   ⎥
⎢                   ⎥
⎣3⋅cos(1)  0  sin(1)⎦

Jacobiano numérico (float):
⎡      4.0          1.0          0         ⎤
⎢                                           ⎥
⎢       0           3.0         2.0        ⎥
⎢                                           ⎥
⎣1.62090691760442    0    0.841470984807897⎦
```

La función arctan

- La función arctan en numpy: Calcula el arcotangente de un número. Devuelve el ángulo en radianes que estará en el intervalo $-\pi/2$ a $\pi/2$.

```
import numpy as np

# Calcular el arcotangente de un valor

y = 1

angulo = np.arctan(y)

print("Ángulo (en radianes) para arctan:", angulo)

print("Ángulo (en grados) para arctan:", np.degrees(angulo))
```

```
Ángulo (en radianes) para arctan: 0.7853981633974483
Ángulo (en grados) para arctan: 45.0
```

La función atan

- La función atan en sympy: Calcula el arcotangente de un valor simbólico o número. Devuelve el ángulo en radianes que estará en el intervalo $-\pi/2$ a $\pi/2$.

```
import sympy as sp
# Definir la variable simbólica
y = sp.Symbol('y')
# Calcular el arcotangente simbólicamente
angulo_simb = sp.atan(y)
# Evaluar el ángulo para y = 1
angulo_val = angulo_simb.subs(y, 1)
# Convertir el ángulo a grados
angulo_degrees = sp.deg(angulo_val)
# Imprimir el resultado simbólico
print("Ángulo simbólico para atan(y):", angulo_simb)
# Imprimir el resultado numérico en radianes
print("Ángulo (en radianes) para atan(1):", angulo_val)
# Imprimir el resultado numérico en grados
print("Ángulo (en grados) para atan(1):", sp.N(angulo_degrees,4))
```

```
Ángulo simbólico para atan(y): atan(y)
Ángulo (en radianes) para atan(1): pi/4
Ángulo (en grados) para atan(1): 45.00
```

La función arctan2

- La función atan2 en numpy: Calcula el ángulo en radianes entre el eje x positivo y el punto (y , x). Devuelve el ángulo en radianes que estará en el intervalo −π a π.

```
import numpy as np
# Definir dos valores. Cuadrante I
y = 1
x = 1
angulo = np.arctan2(y, x)
# Imprimir el resultado
print("Ángulo (en grados) para arctan2(y, x):", np.degrees(angulo))
# Definir dos valores. Cuadrante II
y = 1
x = -1
angulo = np.arctan2(y, x)
print("Ángulo (en grados) para arctan2(y, x):", np.degrees(angulo))
# Definir dos valores. Cuadrante III
y = -1
x = -1
angulo = np.arctan2(y, x)
print("Ángulo (en grados) para arctan2(y, x):", np.degrees(angulo))
# Definir dos valores. Cuadrante IV
y = -1
x = 1
angulo = np.arctan2(y, x)
print("Ángulo (en grados) para arctan2(y, x):", np.degrees(angulo))
```

```
Ángulo (en grados) para arctan2(y, x): 45.0
Ángulo (en grados) para arctan2(y, x): 135.0
Ángulo (en grados) para arctan2(y, x): -135.0
Ángulo (en grados) para arctan2(y, x): -45.0
```

La función atan2

- La función atan2 en sympy:

```
import sympy as sp
# Definir las variables simbólicas
x, y = sp.symbols('x y')
# Calcular el ángulo usando atan2
angulo_simb = sp.atan2(y, x)
# Evaluar el ángulo para y = 1 y x = -1
angulo_val = angulo_simb.subs({y: 1, x: -1})
# Convertir el ángulo a grados
angulo_degrees = sp.deg(angulo_val)
# Imprimir el resultado simbólico
print("Ángulo simbólico para atan2(y, x):", angulo_simb)
# Imprimir el resultado numérico en radianes
print("Ángulo (en radianes) para atan2(1, -1):", angulo_val)
# Imprimir el resultado numérico en grados
print("Ángulo (en grados) para atan2(1, -1):", sp.N(angulo_degrees,5))
```

```
Ángulo simbólico para atan2(y, x): atan2(y, x)
Ángulo (en radianes) para atan2(1, -1): 3*pi/4
Ángulo (en grados) para atan2(1, -1): 135.00
```

Integral indefinida

- La integral indefinida en sympy:

```
La función es: sin(x)**2*cos(x)**2/(sin(x)**4 + cos(x)**4)
La integral es: -x/2 + sqrt(2)*atan(sqrt(2)*sin(x)/cos(x) - 1)/4 +
sqrt(2)*atan(sqrt(2)*sin(x)/cos(x) + 1)/4
La integral evaluada en x = pi/4 es: 0.162661285571072
```

```
import sympy as sp
# Definir la variable simbólica
x = sp.symbols("x")
# Definir la función
f = (sp.sin(x)**2 * sp.cos(x)**2) / (sp.sin(x)**4 + sp.cos(x)**4)
# Calcular la integral de la función
integral = sp.integrate(f, x)
# Imprimir la función y la integral
print("La función es:", f)
print("La integral es:", integral)
# Reemplazar x con un valor específico, por ejemplo, x = pi/4
valor_reemplazo = sp.pi / 4
integral_evaluada = integral.subs(x, valor_reemplazo).evalf()
# Imprimir el resultado numérico
print("La integral evaluada en x =", valor_reemplazo, "es:", integral_evaluada)
```

Integral doble indefinida

- La integral doble indefinida en sympy:

```
import sympy as sp
# Definir las variables simbólicas
x, y = sp.symbols('x y')
# Definir la función
f = x / (x + y)
# Calcular la integral doble indefinida dy dx
integral_dy_dx = sp.integrate(sp.integrate(f, y), x)
# Calcular la integral doble indefinida dx dy
integral_dx_dy = sp.integrate(sp.integrate(f, x), y)
# Imprimir los resultados simbólicos
print("Integral doble indefinida con respecto a y y luego x (simbólica):")
sp.pprint(integral_dy_dx)
print("\nIntegral doble indefinida con respecto a x y luego y (simbólica):")
sp.pprint(integral_dx_dy)
# Reemplazar x = 1 y y = 2 y evaluar el resultado numérico
integral_dy_dx_numerica = integral_dy_dx.subs({x: 1, y: 2}).evalf()
integral_dx_dy_numerica = integral_dx_dy.subs({x: 1, y: 2}).evalf()
# Imprimir los resultados numéricos
print("\nResultado de la integral doble con respecto a y y luego x, con x=1 y y=2:")
print(integral_dy_dx_numerica)
print("\nResultado de la integral doble con respecto a x y luego y, con x=1 y y=2:")
print(integral_dx_dy_numerica)
```

```
Integral doble indefinida con respecto a y y luego x (simbólica):
 2               2          2
x ·log(x + y)   x    x·y   y ·log(x + y)
──────────── - ── + ─── - ──────────
     2          4    2         2

Integral doble indefinida con respecto a x y luego y (simbólica):
 2                    2              2
x ·log(x + y)   x·y   y ·log(x + y)   y
──────────── + ─── - ──────────── + ──
     2          2         2         4

Resultado de la integral doble con respecto a y y luego x, con x=1 y y=2:
-0.897918433002165

Resultado de la integral doble con respecto a x y luego y, con x=1 y y=2:
0.352081566997835
```

Integral definida

- La integral doble definida en sympy:

$$\int_0^1 x^2\,dx = \frac{1}{3}$$

```
import sympy as sp
# Definir la variable simbólica
x = sp.symbols('x')
# Definir la función que queremos integrar
f = x**2
# Calcular la integral definida en el intervalo [0, 1]
integral_definida = sp.integrate(f, (x, 0, 1))
# Evaluar el resultado numéricamente
resultado_numerico = integral_definida.evalf()
# Imprimir el resultado numérico
print("Resultado numérico de la integral definida:")
print(resultado_numerico)
```

```
Resultado numérico de la integral definida:
0.333333333333333
```

Integral doble definida

- La integral doble indefinida en sympy:

$$\int_0^1 \int_0^2 (x^2 + y^2)\, dy\, dx$$

```
Resultado numérico de la integral doble definida:
3.33333333333333
```

```
import sympy as sp
# Definir las variables simbólicas
x, y = sp.symbols('x y')
# Definir la función que queremos integrar
f = x**2 + y**2
# Calcular la integral doble en el intervalo x en [0, 1] y y en [0, 2]
integral_doble = sp.integrate(sp.integrate(f, (y, 0, 2)), (x, 0, 1))
# Evaluar el resultado numéricamente
resultado_numerico = integral_doble.evalf()
# Imprimir el resultado numérico
print("Resultado numérico de la integral doble definida:")
print(resultado_numerico)
```

Integral doble definida

- La integral doble definida en sympy:

$$\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} e^{-(x^2+y^2)}\,dy\,dx$$

```
import sympy as sp
# Definir las variables simbólicas
x, y = sp.symbols('x y')
# Definir la función que queremos integrar
f = sp.exp(-(x**2 + y**2))
# Calcular la integral doble en el intervalo x en [-∞, ∞] y y en [-∞, ∞]
integral_doble = sp.integrate(sp.integrate(f, (y, -sp.oo, sp.oo)), (x, -sp.oo, sp.oo))
# Imprimir el resultado simbólico
print("Resultado simbólico de la integral doble definida:")
sp.pprint(integral_doble)
# Evaluar el resultado numéricamente
resultado_numerico = integral_doble.evalf()
# Imprimir el resultado numérico
print("\nResultado numérico de la integral doble definida:")
print(resultado_numerico)
```

```
Resultado simbólico de la integral doble definida:
π

Resultado numérico de la integral doble definida:
3.14159265358979
```

Integral doble definida

- La integral doble definida en sympy:

$$\int_0^3 \int_{4x/3}^{\sqrt{25-x^2}} x\,dy\,dx$$

```
import sympy as sp
# Definir las variables simbólicas
x, y = sp.symbols("x y")
# Definir la función que queremos integrar
f = x
# Calcular la integral doble en el intervalo y en [4*x/3, sqrt(25 - x**2)] y x en [0, 3]
integral_doble = sp.integrate(sp.integrate(f, (y, 4*x/3, sp.sqrt(25 - x**2))), (x, 0, 3))
# Imprimir la función
print("La función es:", f)
# Imprimir el resultado simbólico de la integral definida
print("\nLa integral definida es:")
sp.pprint(integral_doble)
# Evaluar el resultado numéricamente
resultado_numerico = integral_doble.evalf()
# Imprimir el resultado numérico
print("\nResultado numérico de la integral definida:")
print(resultado_numerico)
```

```
La función es: x

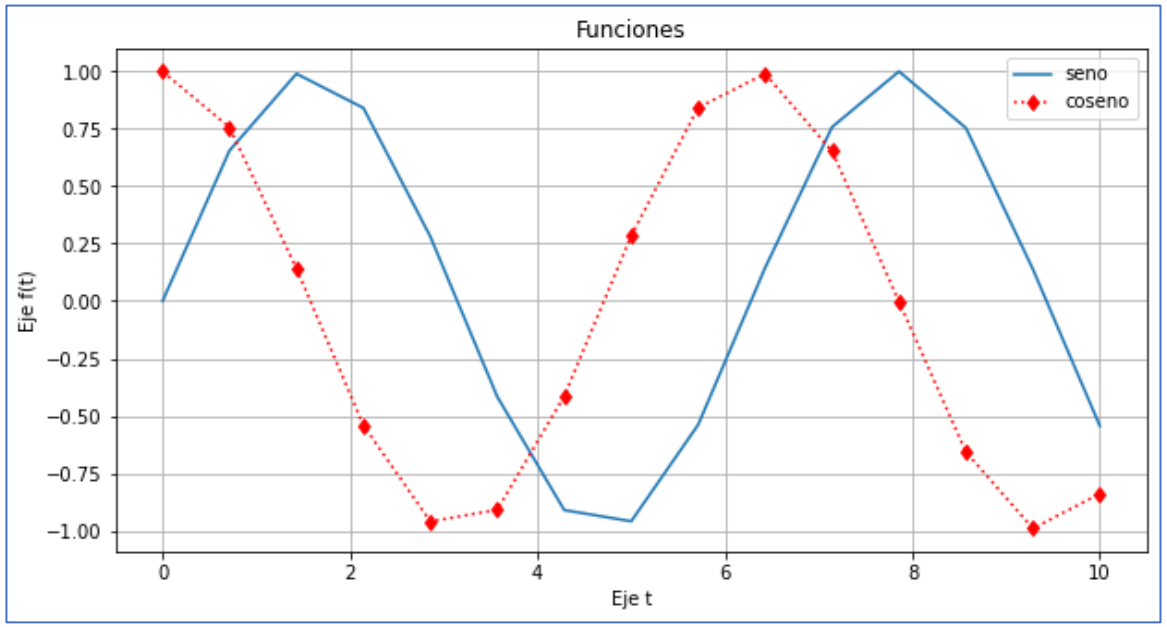
La integral definida es:
25/3

Resultado numérico de la integral definida:
8.33333333333333
```

Gráfica de una función

- Gráfica de la función seno y coseno:

```
import matplotlib.pyplot as plt
import numpy as np
# Definimos el tiempo como un array de valores
t = np.linspace(0, 10, 15) # 15 valores de 0 a 10
f1 = np.sin(t)
f2 = np.cos(t)
plt.figure(figsize=(10,5)) # Tamaño del lienzo en pulgadas
plt.plot(t, f1, label="seno")
plt.plot(t, f2,marker="d",linestyle=":",color="red",label="coseno")
plt.legend()
plt.title("Funciones")
plt.xlabel("Eje t")
plt.ylabel("Eje f(t)")
plt.grid(True)
plt.show()
```



marker: “o” (circulo), “v” (triángulo hacia abajo), “^” (triángulo hacia arriba), “<“ (triángulo hacia la izquierda”), ”>” (triángulo hacia la derecha), “s” (cuadrado), “p” (pentagono), “*” (estrella), “h” (exagono), “+” (cruz), “d” (rombo), etc.

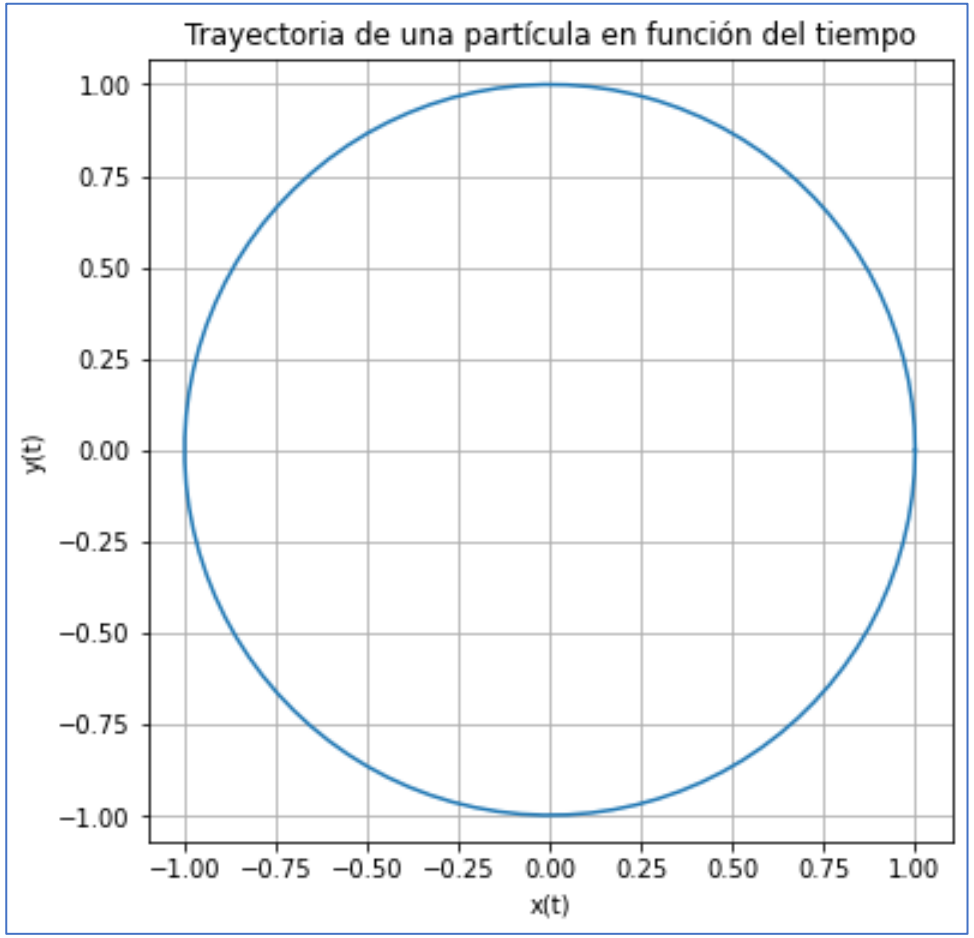
linestyle: “--” (guion), “:” (puntos), “-.” (guion y puntos), “None” (sin línea).

color: “blue”, “green”, “red”, “cyan”, “magenta”, “yellow”, “black”, “white”, o sus iniciales “k” para el negro.

Gráfica de una trayectoria 2D

- Gráfica de una trayectoria en el plano:

```
import numpy as np
import matplotlib.pyplot as plt
# Definimos el tiempo como un array de valores
t = np.linspace(0, 2 * np.pi, 100)  # 100 valores de 0 a 2π
# Definimos las funciones de la trayectoria
x = np.cos(t)  # Coordenada x
y = np.sin(t)  # Coordenada y
# Crear la gráfica
plt.figure(figsize=(6,6)) # Tamaño del lienzo en pulgadas
plt.plot(x, y)
plt.xlabel('x(t)')
plt.ylabel('y(t)')
plt.title('Trayectoria de una partícula en función del tiempo')
plt.grid(True)
plt.axis('equal')  # Asegura que los ejes tengan la misma escala
plt.show()
```



Gráfica de una trayectoria 2D

- Gráfica de una trayectoria en el plano:

```
import numpy as np
import matplotlib.pyplot as plt
# Definimos el rango de valores de x
x = np.linspace(-1, 1, 400)  # 400 valores de -1 a 1
# Calculamos las funciones de y en función de x
y_pos = np.sqrt(1 - x**2)  # Parte positiva de la raíz
y_neg = -np.sqrt(1 - x**2)  # Parte negativa de la raíz
# Crear la gráfica
plt.figure(figsize=(6,6))  # Tamaño del lienzo en pulgadas
plt.plot(x, y_pos, label='y = sqrt(1 - x^2)')
plt.plot(x, y_neg, label='y = -sqrt(1 - x^2)')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Trayectoria de una partícula y versus x')
plt.grid(True)
plt.axis('equal')  # Asegura que los ejes tengan la misma escala
plt.legend()
plt.show()
```

- $x = \cos(t)$
- $y = \sin(t)$

Podemos utilizar la identidad trigonométrica fundamental:

$$\sin^2(t) + \cos^2(t) = 1$$

Dado que $x = \cos(t)$, entonces $\cos^2(t) = x^2$. Y como $y = \sin(t)$, entonces $\sin^2(t) = y^2$.

Aplicamos la identidad trigonométrica:

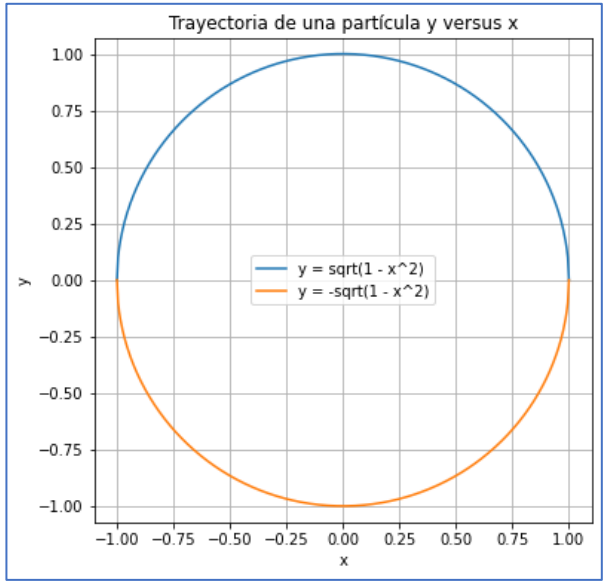
$$\sin^2(t) + \cos^2(t) = 1$$

Sustituyendo:

$$y^2 + x^2 = 1$$

Por lo tanto, la ecuación de la trayectoria es una circunferencia con radio 1, y la función explícita es:

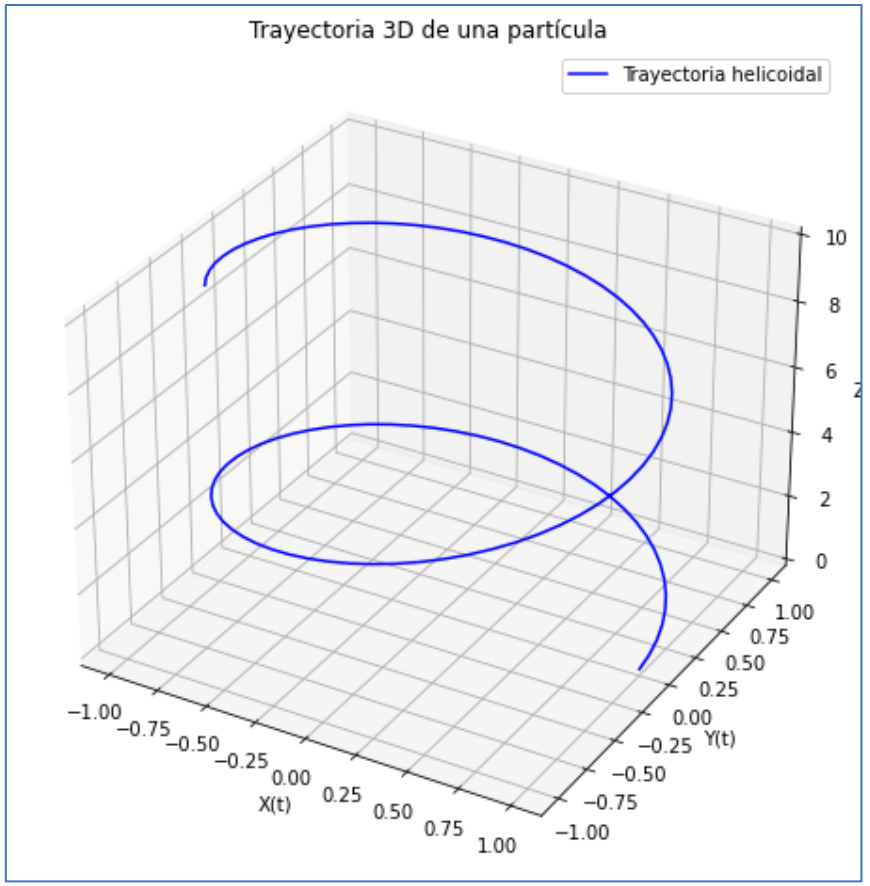
$$y = \pm\sqrt{1 - x^2}$$



Gráfica de una trayectoria 3D

- Gráfica de una trayectoria en el espacio:

```
import numpy as np
import matplotlib.pyplot as plt
# Definir el tiempo como un array de valores
t = np.linspace(0, 10, 100)  # 100 valores de 0 a 10
# Definir las funciones de la trayectoria
x = np.cos(t)  # Coordenada x
y = np.sin(t)  # Coordenada y
z = t          # Coordenada z
# Crear una figura y un gráfico 3D
fig = plt.figure(figsize=(10,8)) # Lienzo
ax = fig.add_subplot(projection='3d') # Crea un espacio en el lienzo para dibujar en 3D
# Graficar la trayectoria
ax.plot(x, y, z, label='Trayectoria helicoidal', color='b')
ax.set_xlabel('X(t)')
ax.set_ylabel('Y(t)')
ax.set_zlabel('Z(t)')
ax.set_title('Trayectoria 3D de una partícula')
ax.legend()
# Mostrar la gráfica
plt.show()
```



FIN